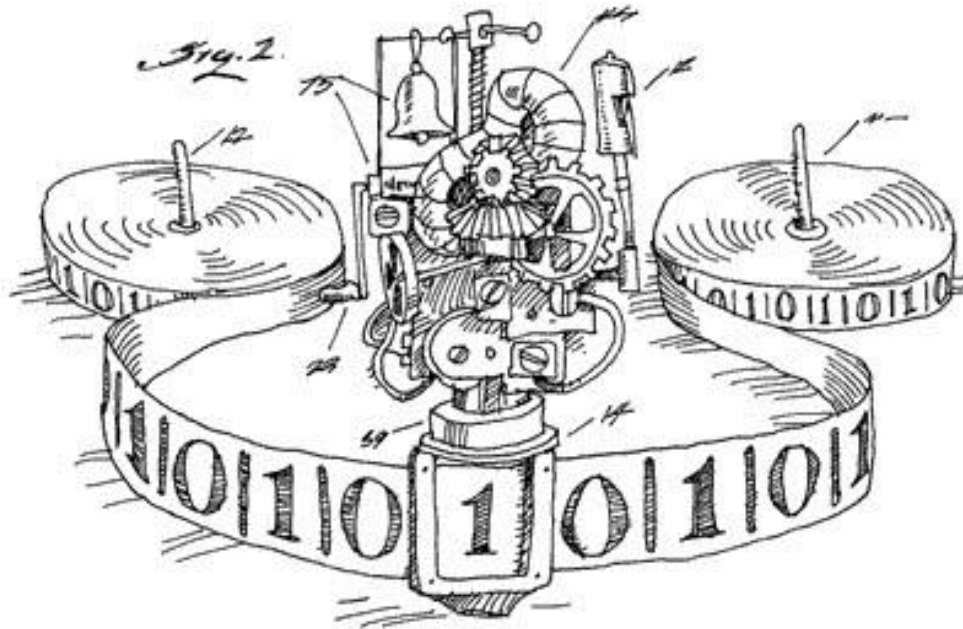




Teoría de la Computación

2025

Dr. Rubén Alfredo Bernal

Docentes

Teoría

- Dr. Rubén Alfredo Bernal
 - rbernal@exa.unne.edu.ar

Práctico

- Dr. Emanuel Irrazábal
 - eirrazabal@exa.unne.edu.ar

Bibliografía

- Hopcroft, J. E.; Motwani, R.; Ullman, J. D. *Introducción a la teoría de autómatas, lenguajes y computación* (3ra. Ed.). Pearson, 2007.
- Sipser, M. *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning, 2012.
- Brookshear, J. G. *Teoría de la computación - Lenguajes formales, autómatas y complejidad*. Addison-Wesley, 1993.

Horarios de clases

- **Clases teóricas**
 - **Viernes (18:00 – 20:00)**
- **Clases prácticas**
 - **Lunes 17:00**

Instancias de Evaluación

Actividad	Fecha
Primer parcial	12/05
Segundo parcial	16/06
Recuperatorio del primer parcial	23/06
Recuperatorio del segundo parcial	23/06
Recuperatorio extraordinario	30/06
Presentación del trabajo integrador	27/06 y 04/07

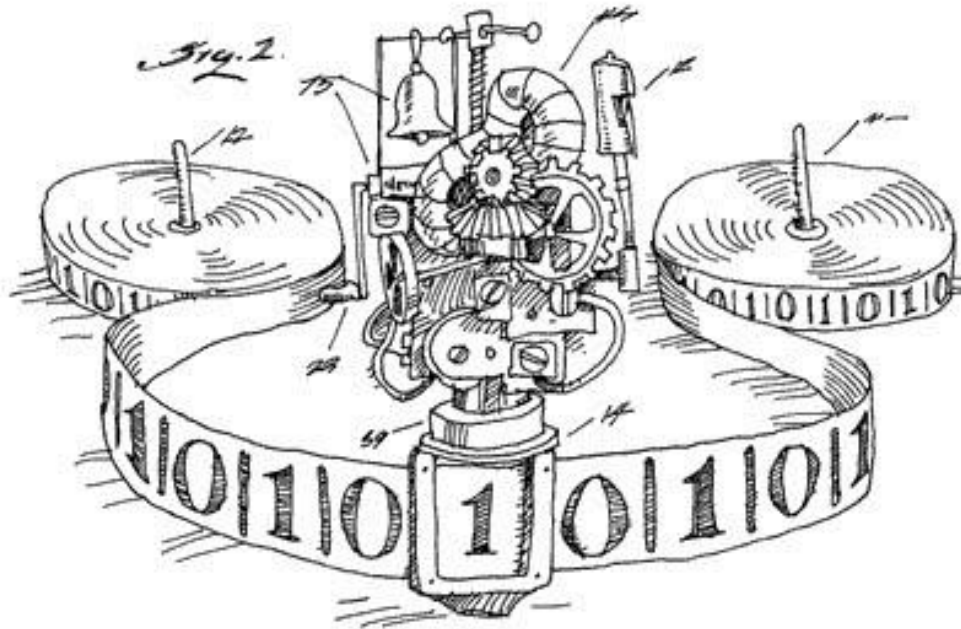
Condiciones de regularidad

- **Para Regularizar**

- Los 2 parciales aprobados con nota no inferior a 6 (seis) con opción a dos recuperatorios más extraordinario.
- Trabajo integrador aprobado.

- **Para Promocionar**

- Los 2 parciales aprobados con nota no inferior a 7 (siete) con opción a un recuperatorio de cualquiera de los dos parciales (cuya nota haya sido mayor a 5).
- Trabajo integrador aprobado.



Teoría de la Computación

Introducción



Contenidos generales

- Introducción. Teoría de la computación
- Lenguajes y Gramáticas
- Aceptores de Estados Finitos
- Aceptores Push Down
- Máquina de Turing
- Computabilidad y Complejidad

Algoritmo

The definition of algorithm came in the 1936 papers of Alonzo Church and Alan Turing. Church used a notational system called the λ -calculus to define algorithms. Turing did it with his “machines.” These two definitions were shown to be equivalent.

Algoritmo

Secuencia ordenada de pasos para resolver un problema

Preciso: debe tener una secuencia de pasos definidos de una manera clara.

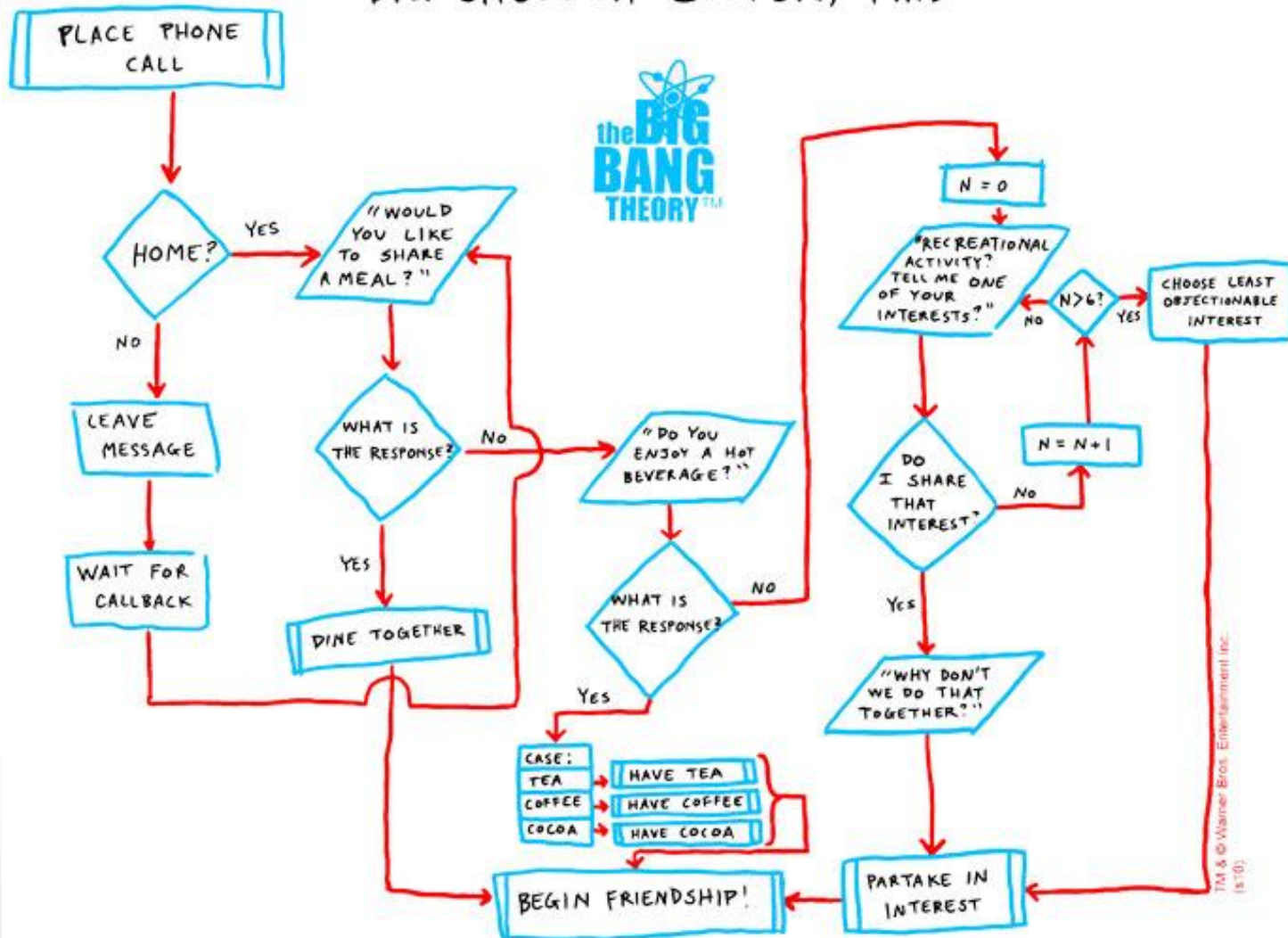
Finito: Debe terminar en algún momento.

Definido: Siempre que se sigan los pasos, debe arrojar la misma salida.



THE FRIENDSHIP ALGORITHM

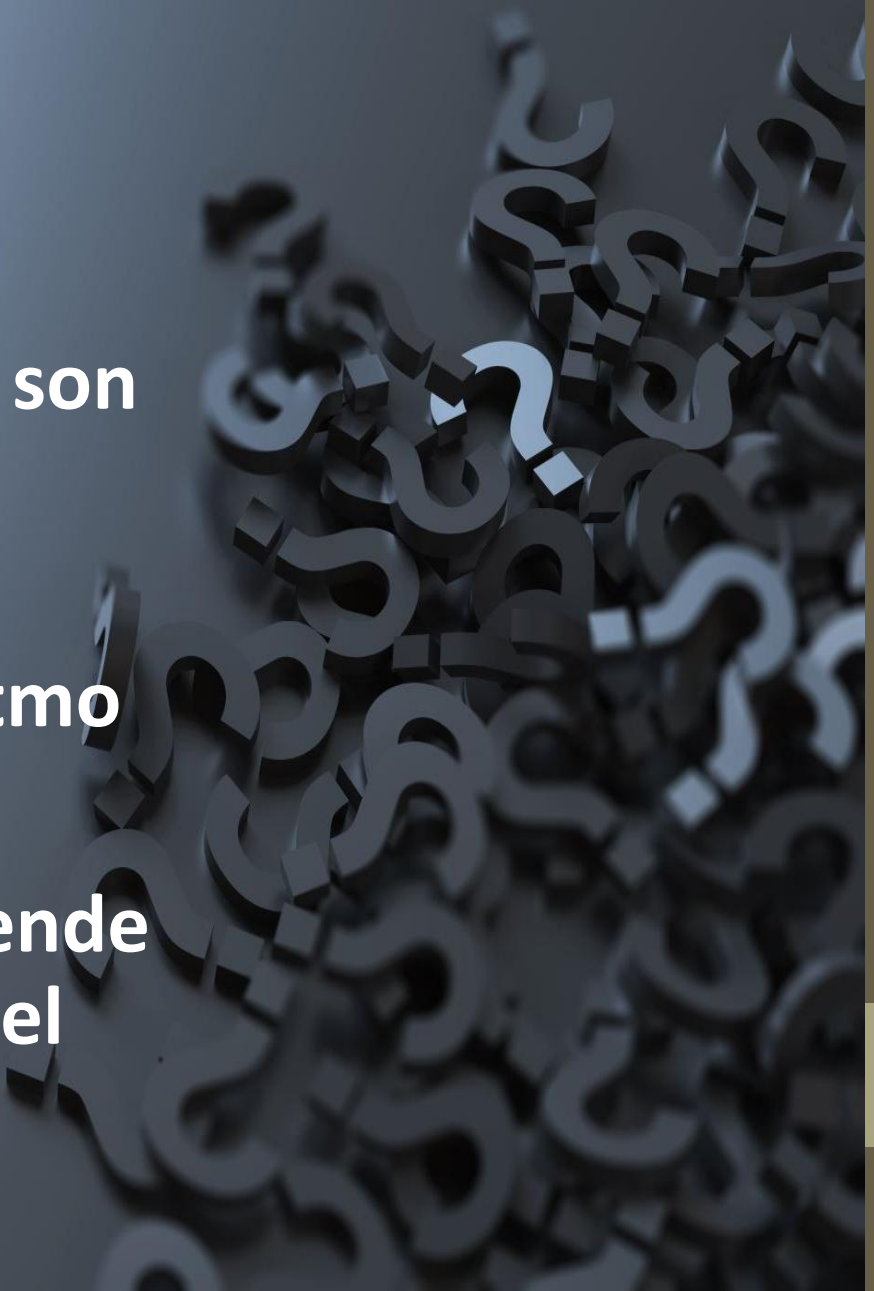
DR. SHELDON COOPER, Ph.D



¿Todos los problemas son computables?

Computable = Algoritmo

¿Ser computable depende de la complejidad del problema?



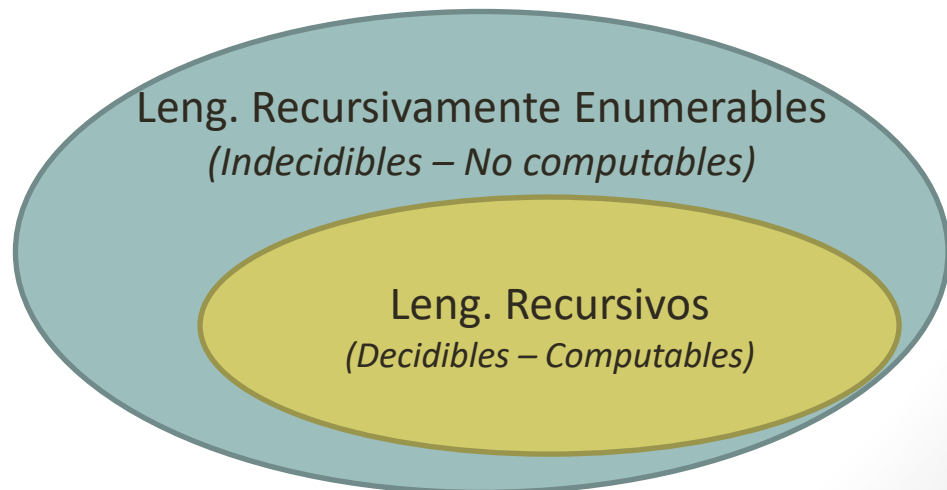
Problema

Un problema es computable si existe un algoritmo (o una MT) que lo resuelva.

Decidibles o Computables: La MT se detiene aceptando para cadenas que pertenecen al lenguaje y se detiene en estado de rechazo para cadenas que no pertenecen al lenguaje.

Indecidibles o no Computables: Existe una MT que, si la cadena pertenece al lenguaje, la MT se detiene aceptándola, si la cadena no pertenece a lenguaje, la MT no se detiene.

El lenguaje es L_{re}



https://es.wikipedia.org/wiki/Problema_indecidible#Ejemplos_de_problemas_indecidibles

Ciencias de la Computación



Teoría de
Lenguajes y
gramáticas



Teoría de la
Computabilidad
y Complejidad



Teoría de
Máquinas

Marco Histórico

- 1928 – David Hilbert. Las matemáticas son completas, consistentes y decidibles. Problema de decisión.
- 1928 – Ackermann. Problema de decisión en el estudio sobre la lógica de primer orden.
- 1930 – Kurt Godel. Teorema de la incompletitud.
- 1936 – Alonzo Church. Cálculo Lambda. Funciones recursivas.
- 1936 – Alan Turing. Máquina de Turing. Basado en el concepto de algoritmo. Estudio de la computabilidad. Máquina Universal.
- 1954 – George Mealy. Estudio sobre autómatas.
- 1956 – Edward Moore. Estudio sobre autómatas.
- 1956 – Noam Chomsky. Estudio sobre gramáticas y lenguajes.
- 1969 – Stephen Cook. Continúa el estudio sobre la complejidad. Relación entre las clases de complejidad P y NP.

Jerarquía de Chomsky

Tipo	Gramática		Lenguaje Aceptado	Autómata	
Type 0	No restringidas	→ Lenguajes más restringidos →	Lenguaje recursivamente enumerable	Turing Machine	→ Máquinas más poderosas →
Type 1	Sensibles al contexto		Lenguajes sensibles al contexto	Linear-bounded automaton	
Type 2	Libres de contexto		Lenguajes libres de contexto	Pushdown automaton	
Type 3	Regulares		Lenguajes regulares	Finite state automaton	

¿Dónde consultar?

- Michael Sipser
 - Strings and Languages – pag 13

Strings y Lenguajes

- **Alfabeto:** Conjunto finito de símbolos que representa los objetos o elementos atómicos que se emplean en la construcción de sentencias (Σ).
- **String de longitud k :** Es una secuencia de k símbolos en Σ .

$$\omega = v_1 v_2 v_3 \dots v_k$$

$$|\omega| = k$$

- **String vacío:** Es el único string que no contiene símbolos y es indicado con la letra λ .

$$|\lambda| = 0$$

- **Reverso de un string:** el reverso de $\omega = v_1 v_2 v_3 \dots v_k$ es otro string $\omega^R = v_k \dots v_3 v_2 v_1$

Conjuntos W_k y W

- **Conjunto W_k**

Es el conjunto de todos los strings de longitud k sobre un alfabeto Σ .

$$W_k = \{ \omega / |\omega| = k \}$$

- **Conjunto W**

Es el conjunto de todos los strings posibles sobre un alfabeto V .

$$W = \bigcup_{k=0}^{\infty} W_k$$

Concatenación de Strings

Es una operación binaria también denominada multiplicación de strings:

$$m: W \times W \rightarrow W$$

Dados: $\alpha = v_1 v_2 \dots v_p$ y $\beta = u_1 u_2 \dots u_q$

$$m: (\alpha, \beta) = v_1 v_2 \dots v_p u_1 u_2 \dots u_q$$

Propiedades

Es cerrada

Asociativa

Elemento neutro

No conmutativa

Longitud de la concatenación es la suma de las longitudes

Lenguajes

Un lenguaje es cualquier conjunto de strings definidos sobre el alfabeto Σ . Es un subconjunto de W .

Representamos los lenguajes con letras mayúsculas: L, M, N

- **Ejemplos**

Consideramos el alfabeto; $\Sigma = \{a, b\}$

$$L_1 = \{aa, b, bba, \lambda\}$$

$$L_2 = \{\lambda, ab, aabb, aaabbb, \dots\} = \{a^i b^i \mid i \geq 0\}$$

Operaciones con Lenguajes

- **Concatenación**

$$L.M = \{\varphi\psi / \varphi \in L; \psi \in M\}$$

Propiedades (estructura de Monoide):

Elemento Neutro: $L.\{\lambda\} = \{\lambda\}.L = L$

Asociativa: $L.(M.N) = (L.M).N$

Elemento Nulo: $L.\emptyset = \emptyset.L = \emptyset$

Operaciones con Lenguajes

- **Clausura de L**

$$L^* = \bigcup_{k=0}^{\infty} L^k$$

donde $L^0 = \{\lambda\}$

$$L^1 = L$$

$$L^2 = L^1 L$$

$$L^3 = L^2 L$$

...

$$L^k = L^{k-1} L$$

- $L^+ = L^* - \{\lambda\}$

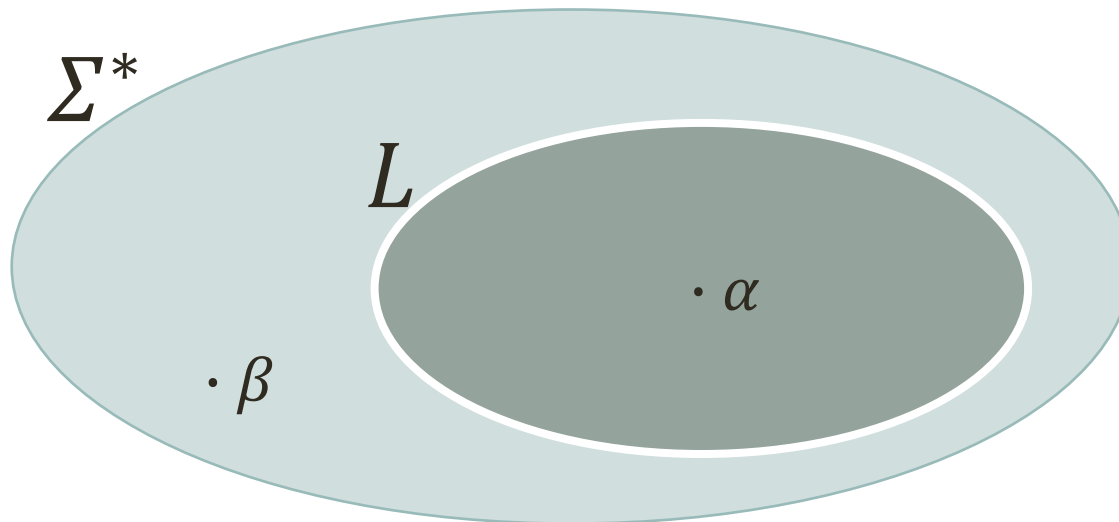
Operaciones con Lenguajes

- Reverso de L

$$L^R = \{\omega^R / \omega \in L\}$$

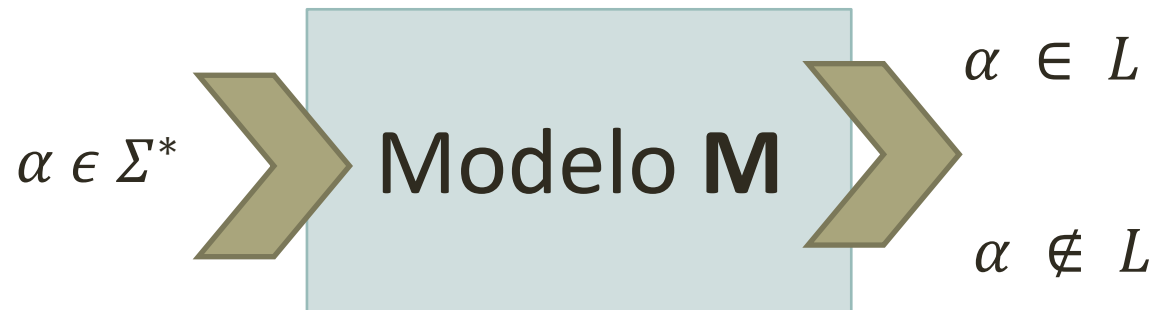
Problema

- Dado un string ω sobre un alfabeto $\Sigma = \{a, b\}$, ¿cómo podemos determinar si el string pertenece o no a un lenguaje L ?



Problema

- Debemos construir un modelo que permita responder ese problema de decisión.

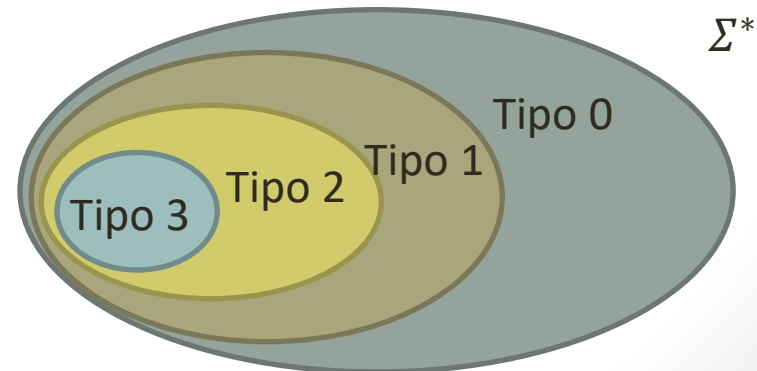


Jerarquía de Lenguajes

Clasificación de Chomsky

Tipo	Lenguaje	Máquina	Gramática
0	Recursivamente enumerable	Máquina de Turing	No restringidas
1	Dependiente del contexto	Autómata linealmente acotado	Sensible al contexto
2	Independiente del contexto	Autómata de pila	Libres de contexto
3	Regular	Autómata de estados finitos	Regular

$$L_3 \subset L_2 \subset L_1 \subset L_0 \subset \Sigma^*$$



Conclusiones

1

La Teoría de la computación está basada en definiciones matemáticas.

2

Los problemas en Teoría de la computación consisten en verificar la pertenencia o no de una cadena a un lenguaje. Se corresponden con un problema de decisión.

3

Un lenguaje es un conjunto de cadenas con determinadas características formadas a partir de un alfabeto.

Gramáticas Formales

Es una representación *generativa* de un lenguaje mediante el uso de una estructura finita compuesta de reglas.

<oración> → <sujeto><predicado>

<sujeto> → <frase nominal>

<frase nominal> → <sustantivo>

<frase nominal> → <sustantivo><conjunción><frase nominal>

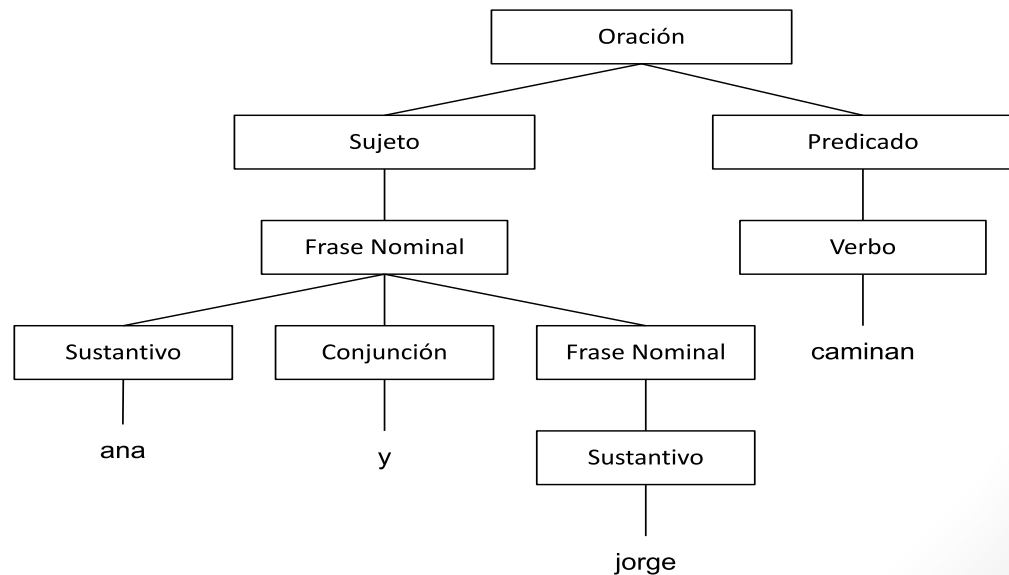
<conjunción> → y

<sustantivo> → ana

<sustantivo> → pablo

<predicado> → <verbo>

<verbo> → caminan



Gramáticas Formales

Podemos realizar un proceso de *generación* de una cadena del lenguaje mediante el uso de una derivación.

$\langle O \rangle \rightarrow \langle S \rangle \langle P \rangle$

$\langle S \rangle \rightarrow \langle FN \rangle$

$\langle FN \rangle \rightarrow \langle U \rangle$

$\langle FN \rangle \rightarrow \langle U \rangle \langle C \rangle \langle FN \rangle$

$\langle C \rangle \rightarrow y$

$\langle U \rangle \rightarrow ana$

$\langle U \rangle \rightarrow pablo$

$\langle P \rangle \rightarrow \langle V \rangle$

$\langle V \rangle \rightarrow caminan$

Definición

$$G = (N, T, P, S)$$

$$N \cap T = \emptyset$$

$$S \notin N \cup T$$

$$\alpha \rightarrow \beta \in P$$

donde

$$\alpha = \varphi A \psi$$

$$\beta = \varphi \omega \psi$$

tal que

$$\varphi, \omega, \psi \in (N \cup T)^*$$

$$A \in (N \cup \{S\})$$

Tipos de Gramáticas

Clasificación de Chomsky

0	$\varphi A \psi \rightarrow \varphi \omega \psi$	<i>No Restringida</i>	
1	$S \rightarrow \lambda$ $\varphi A \psi \rightarrow \varphi \omega \psi$	<i>Sensible al Contexto</i>	
2	$S \rightarrow \lambda$ $A \rightarrow \omega$	<i>Libres de Contexto</i>	
3	$S \rightarrow \lambda$ $A \rightarrow aB$ $A \rightarrow a$	<i>RLD</i>	Regulares Tipo 0 Tipo 1 Tipo 2 Tipo 3
	$S \rightarrow \lambda$ $A \rightarrow Ba$ $A \rightarrow a$	<i>RLI</i>	

Gramáticas Tipo 3

$$\alpha \rightarrow \beta$$

$$\alpha = \varphi A \psi$$

$$\beta = \varphi \omega \psi$$

$$\varphi, \omega, \psi \in (N \cup T)^*$$

$$A \in (N \cup \{S\})$$

$$B \in N$$

$$a \in T$$

$$S \rightarrow \lambda$$

$$A \rightarrow Ba$$

$$A \rightarrow a$$

RLI

$$S \rightarrow \lambda$$

$$A \rightarrow aB$$

$$A \rightarrow a$$

RLD

Gramáticas Tipo 2

$$\alpha \rightarrow \beta$$

$$\alpha = \varphi A \psi$$

$$\beta = \varphi \omega \psi$$

$$\varphi, \omega, \psi \in (N \cup T)^*$$

$$A \in (N \cup \{S\})$$

$$S \rightarrow \lambda$$

$$A \rightarrow \omega$$

$$\omega \neq \lambda$$

Gramáticas Tipo 1

$$\alpha \rightarrow \beta$$

$$\alpha = \varphi A \psi$$

$$\beta = \varphi \omega \psi$$

$$\varphi, \omega, \psi \in (N \cup T)^*$$

$$A \in (N \cup \{S\})$$

$$S \rightarrow \lambda$$

$$\varphi A \psi \rightarrow \varphi \omega \psi$$

$$\omega \neq \lambda$$

Gramáticas Tipo 0

$$\alpha \rightarrow \beta$$

$$\alpha = \varphi A \psi$$

$$\beta = \varphi \omega \psi$$

$$\varphi, \omega, \psi \in (N \cup T)^*$$

$$A \in (N \cup \{S\})$$

$$\varphi A \psi \rightarrow \varphi \omega \psi$$

Forma sentencial

- Un string de símbolos en $(N \cup T)^* \cup \{S\}$ se conoce como forma sentencial.

- Derivación inmediata de ω en ω'

$$\omega \Rightarrow \omega'$$

- Derivación de ω_n a partir de ω_1

$$\omega_1 \xRightarrow{*} \omega_n$$

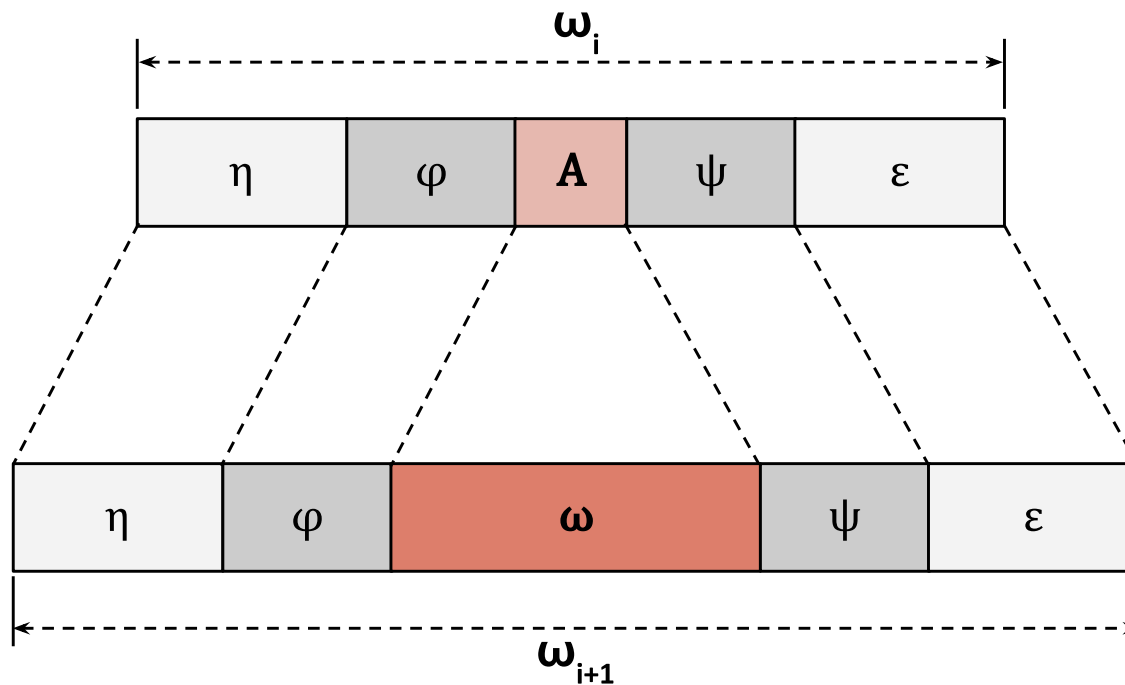
Lenguaje generado

$$L(G) = \left\{ \omega \in T^* / S \stackrel{*}{\Rightarrow} \omega \right\}$$

Si $\omega \in L(G)$, decimos que ω es un string, una sentencia o una palabra en el lenguaje generado por G .

Derivación de ω_{i+1} a partir de ω_i

Aplicación de la producción: $\varphi A \psi \rightarrow \varphi \omega \psi$

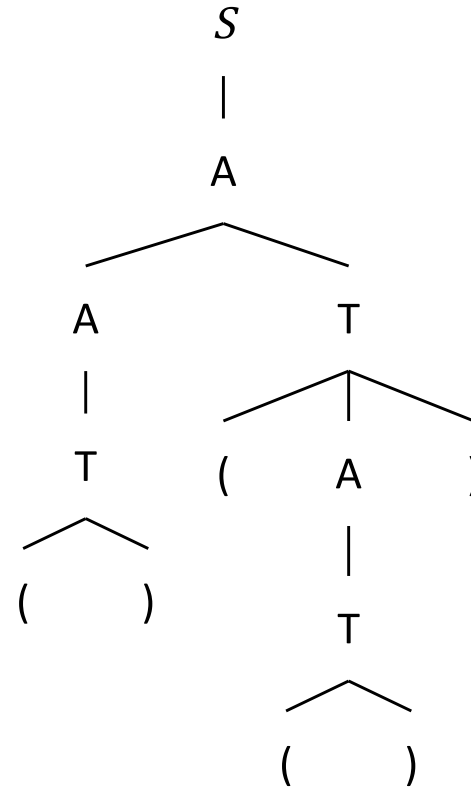


Árboles de derivación

Dada la siguiente gramática G que genera el lenguaje de paréntesis (L_p), encontrar un *árbol de derivación* para el string $()()$

$G:$

- $S \rightarrow A$
- $A \rightarrow AT$
- $A \rightarrow T$
- $T \rightarrow (A)$
- $T \rightarrow ()$



Derivaciones

Dada la siguiente gramática G que genera el lenguaje de paréntesis (L_p), encontrar una *derivación* para el string $()()$

$G:$

- $S \rightarrow A$
- $A \rightarrow AT$
- $A \rightarrow T$
- $T \rightarrow (A)$
- $T \rightarrow ()$

Derivaciones por izquierda

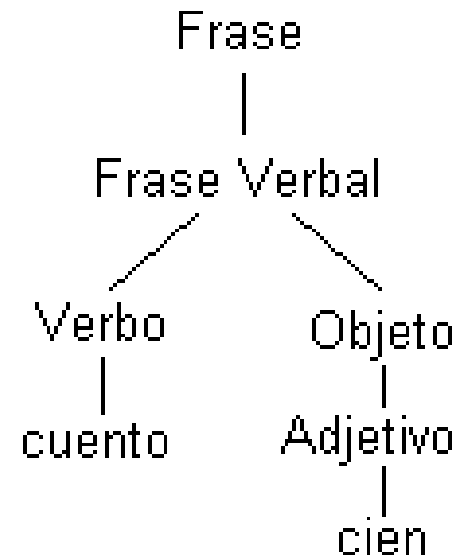
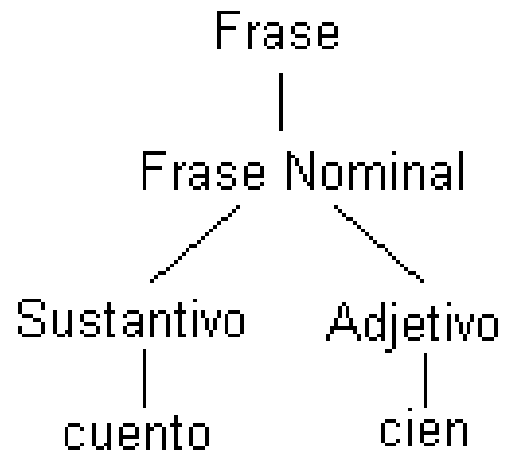
- Sea G una gramática libre de contexto. Una derivación $\omega_1 \Rightarrow \omega_2 \Rightarrow \omega_2 \Rightarrow \cdots \Rightarrow \omega_n$ es una *derivación por izquierda* sí y sólo sí el símbolo no terminal ubicado más a la izquierda de ω_i es reemplazado para poder obtener ω_{i+1} para $0 \leq i < n$.
- Esto es:
Partiendo de $\omega_i = \alpha A \beta$ obtenemos $\omega_{i+1} = \alpha \omega \beta$
con $\alpha \in T^*$, $A \in N$ y $\beta \in (N \cup T)^*$
y $A \rightarrow \omega$ es una producción definida en G .

Ambigüedad

- Sea G una gramática libre de contexto y sea ω una sentencia en el lenguaje generado por G . Luego, ω es ambiguo si existen derivaciones de ω que se corresponden con árboles de derivación diferentes.
- Una gramática libre de contexto G es ambigua sí y sólo si genera una sentencia dada a partir de dos o más derivaciones por izquierda distintas.

Lenguaje Natural.

Ejemplo de ambigüedad



Ambigüedad. Tipos

- **Estructural**

$$\begin{aligned}G_1: \quad & S \rightarrow A \\ & A \rightarrow A\emptyset A \\ & A \rightarrow 1\end{aligned}$$

- Armar árboles para 10101

- **Asociada a Rótulos**

$$\begin{aligned}G_2: \quad & S \rightarrow A \\ & A \rightarrow B\emptyset \\ & A \rightarrow A\emptyset \\ & B \rightarrow B\emptyset \\ & A \rightarrow 1 \\ & B \rightarrow 1\end{aligned}$$

- Armar árboles para 100